

PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

Nama GLORY HARDY FEBRIANDO SIANTURI

NIM 105223013

SOAL 1

```
class Vehicle {  
    void speedUp() { System.out.println("Vehicle accelerating"); }  
}  
  
class Car extends Vehicle {  
    @Override  
    void speedUp() { System.out.println("Car accelerating by 22 units"); }  
  
    void drift() { System.out.println("Performing a drift"); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Vehicle myVehicle = new Car(); // Upcasting  
        myVehicle.speedUp();  
        // myVehicle.drift(); // Baris ini menyebabkan error "cannot find symbol"  
    }  
}
```

Pertanyaan 1: Mengapa metode drift() tidak dapat dipanggil melalui variabel myVehicle padahal objek aktualnya adalah Car?

Jawaban:

Hal ini terjadi karena konsep Tipe Referensi (Reference Type) vs Tipe Objek (Object Type) di Java.

Ketika Anda mendeklarasikan `Vehicle myVehicle = new Car();`, variabel `myVehicle` memiliki tipe referensi `Vehicle`, meskipun objek aktual yang dibuat di memori adalah `Car`.

Pada saat proses kompilasi (compile-time), *compiler* Java menentukan metode apa saja yang boleh dipanggil hanya berdasarkan tipe referensinya, bukan objek aktualnya. Karena metode `drift()` hanya ada di kelas `Car` dan tidak didefinisikan di dalam kelas `Vehicle`, maka *compiler* akan menampilkan error "cannot find symbol". *Compiler* tidak melihat bahwa objek tersebut sebenarnya adalah `Car` sampai program benar-benar dijalankan (runtime).

Pertanyaan 2: Tuliskan blok kode perbaikan menggunakan operator `instanceof` dan Downcasting yang aman untuk memanggil `drift()`.

Jawaban: Untuk memanggil metode `drift()`, kita perlu mengubah kembali referensi `Vehicle` menjadi `Car` (Downcasting). Untuk memastikan proses ini aman dan tidak memicu error `ClassCastException`, kita harus mengeceknya terlebih dahulu menggunakan `instanceof`.

```
public class Main {  
    public static void main(String[] args) {  
        Vehicle myVehicle = new Car(); // Upcasting  
        myVehicle.speedUp();  
  
        // Pengecekan dengan instanceof dan Downcasting yang aman  
        if (myVehicle instanceof Car) {  
            Car myCar = (Car) myVehicle; // Downcasting  
            myCar.drift();                // Sekarang metode drift() bisa dipanggil  
        }  
    }  
}
```

Soal 2

```
class OverloadScenario {  
  
    void test(int a, double b) { System.out.println("Fase A: int, double"); }  
  
    void test(double a, int b) { System.out.println("Fase B: double, int"); }  
  
}
```

Pertanyaan Analisis: Jika Anda mengeksekusi `example.test(10, 5)`; apakah kode tersebut akan berjalan? Jelaskan mengapa Java Compiler menganggap pemanggilan tersebut ambigu?

Jawaban:

Tidak, kode tersebut tidak akan berjalan. Jika Anda mencoba mengkompilasinya, akan muncul *compile-time error* yang menyatakan bahwa referensi ke metode test tersebut ambigu (*reference to test is ambiguous*).

Alasan mengapa Java Compiler menganggapnya ambigu:

Ketika pemanggilan `example.test(10, 5)`; dilakukan, Anda mengirimkan dua argumen berupa angka bulat. Secara default, angka 10 dan 5 dibaca oleh Java sebagai tipe data `int`.

Java Compiler kemudian mencari metode yang paling cocok (*most specific method*) dari dua opsi yang tersedia. Ia akan mencoba mencocokkannya melalui proses yang disebut *Implicit Type Casting* atau *Widening* (memperluas tipe data dari `int` ke `double`):

1. Pencocokan ke Metode 1 (`test(int a, double b)`): Argumen pertama 10 (`int`) cocok secara persis. Argumen kedua 5 (`int`) dapat dipromosikan/diperluas dengan aman menjadi `double` (menjadi 5.0). Jadi, metode ini valid.
2. Pencocokan ke Metode 2 (`test(double a, int b)`): Argumen pertama 10 (`int`) dapat dipromosikan/diperluas dengan aman menjadi `double` (menjadi 10.0). Argumen kedua 5 (`int`) cocok secara persis. Jadi, metode ini juga valid.

Karena kedua metode tersebut sama-sama valid dan masing-masing mengharuskan *compiler* untuk melakukan satu konversi *widening* secara menyilang, *compiler* menjadi bingung. Tidak ada satu metode pun yang secara mutlak lebih spesifik atau lebih cocok daripada yang lain. Untuk mencegah eksekusi yang tidak tertebak, *compiler* menolak untuk memilih dan langsung menghasilkan error ambiguitas.